1. Write a **Rectangle class** in Python language, allowing you to build a rectangle with **length** and **width** attributes.Create a **Perimeter()** method to calculate the perimeter of the rectangle and a **Area()** method to calculate the area of the rectangle.Create a method **display()** that display the length, width, perimeter and area of an object created using an instantiation on rectangle class.Create a **Parallelepipede** child class **inheriting** from the **Rectangle class** and with a **height** attribute and another **Volume()** method to calculate the volume of the **Parallelepiped.**

---

2. Create a Python class **Person** with attributes: **name** and **age** of type string.Create a **display()** method that displays the name and age of an object created via the Person class.Create a **child class Student** which **inherits** from the Person class and which also has a **section** attribute.Create a **method displayStudent()** that displays the name, age and section of an object created via the Student class.Create a **student object** via an instantiation on the Student class and then test the displayStudent method.

---

3. Create a Python class called **BankAccount** which represents a bank account, having as attributes: **accountNumber** (numeric type), **name** (name of the account owner as string type), balance.Create a **constructor** with parameters: **accountNumber, name, balance**.Create a **Deposit()** method which manages the deposit actions.Create a **Withdrawal() method** which manages withdrawals actions.Create an **bankFees()** method to apply the bank fees with a percentage of 5% of the balance account.Create a **display()** method to display account details.Give the complete code for the **BankAccount class**.

---

4. Define a **Book** class with the following attributes: **Title, Author (Full name), Price**.Define a **constructor** used to initialize the attributes of the method with values entered by the user.Set the View() method to display information for the current book.Write a program to testing the Book class.

---

5. Create a class called **TK_extended** which inherits from TK class and having the attributes:

   Master: that represents the name of the main window

   title: that represents the title of the main window

   Create a **method** called **create()** that creates the window

   Create a **method** called **resize(width, height)** that can resize the window.

   Create a **method** called **generate()** to generate the window

---

6. **Create a child class Bus that will inherit all of the variables and methods of the Vehicle class. In the vehicle class create relevant methods and variables.**
7. Define a property that must have the same value for every class instance (object). Define a class attribute "color" with a default value white. I.e., Every Vehicle should be white.
8. Create a class Employee with attributes name, id, salary and methods get_name(), get_id(), get_salary(), set_salary(), and calculate_bonus(). Create an object of the Employee class and demonstrate the use of all the methods.

9. Create a class Rectangle with attributes length and width and methods calculate_area() and calculate_perimeter(). Create a subclass Square that inherits from Rectangle and overrides the calculate_perimeter() method. Create objects of both the classes and demonstrate the use of all the methods.
10. Create a class Vehicle with attributes make, model, and year and methods get_make(), get_model(), get_year(), and set_year(). Create a subclass Car that inherits from Vehicle and adds the attribute num_doors and method get_num_doors(). Create objects of both the classes and demonstrate the use of all the methods.
11. Create a class Shape with attributes color and methods set_color() and get_color(). Create a subclass Rectangle that inherits from Shape and adds attributes length and width, and methods calculate_area() and calculate_perimeter(). Create objects of both the classes and demonstrate the use of all the methods.
12. Create a class BankAccount with attributes account_number, balance, and interest_rate, and methods deposit(), withdraw(), get_balance(), and get_interest_rate(). Create a subclass SavingsAccount that inherits from BankAccount and adds the attribute min_balance and method calculate_interest(). Create objects of both the classes and demonstrate the use of all the methods.
13. Create a class Employee with attributes name, id, salary and methods get_name(), get_id(), get_salary(), set_salary(), and calculate_bonus(). Create a subclass Manager that inherits from Employee and adds the attribute department and method get_department(). Create objects of both the classes and demonstrate the use of all the methods.
14. Create a class BankAccount with attributes account_number, balance, and interest_rate, and methods deposit(), withdraw(), get_balance(), and get_interest_rate(). Create a subclass CheckingAccount that inherits from BankAccount and adds the attribute overdraft_fee and method apply_overdraft_fee(). Create objects of both the classes and demonstrate the use of all the methods.
15. Create a class Shape with attributes color and methods set_color() and get_color(). Create a subclass Circle that inherits from Shape and adds the attribute radius, and methods calculate_area() and calculate_circumference(). Create objects of both the classes and demonstrate the use of all the methods.

16. Create a class Animal with attributes name, species, and age, and methods get_name(), get_species(), get_age(), and make_sound(). Create a subclass Dog that inherits from Animal and adds the attribute breed, and method bark(). Create objects of both the classes and demonstrate the use of all the methods.
17. Create a class Person with attributes name, age, and gender, and methods get_name(), get_age(), get_gender(), and introduce(). Create a subclass Student that inherits from Person and adds the attribute grade, and method study(). Create objects of both the classes and demonstrate the use of all the methods.
18. You are building a game that involves different characters, each with their own set of attributes and abilities. How would you use OOP to model these characters and their interactions in the game?
19. You are building a system to manage a company's inventory, which includes different types of items with their own attributes and behaviors. How would you use OOP to model these items and their interactions in the inventory system?
20. You are building a web application that involves user authentication and authorization, with different roles and permissions. How would you use OOP to model the user roles and their permissions, and how would you enforce these roles and permissions in the application?
21. You are building a system to simulate a traffic network, with different types of vehicles and their movements on the roads. How would you use OOP to model the vehicles, the roads, and their interactions in the traffic simulation?
22. You are building a system to manage a library, with different types of books and their borrowing policies. How would you use OOP to model the books, the borrowers, and their interactions in the library management system?
23. Create a class BankAccount with attributes account_number, balance, and interest_rate, and methods deposit(), withdraw(), get_balance(), and get_interest_rate(). Create a subclass SavingsAccount that inherits from BankAccount and adds the attribute minimum_balance and method calculate_interest(). Create objects of both the classes and demonstrate the use of all the methods.
24. Create a class Vehicle with attributes make, model, and year, and methods get_make(), get_model(), get_year(), and start_engine(). Create a subclass Car that inherits from

Vehicle and adds the attribute num_doors and method lock_doors(). Create objects of both the classes and demonstrate the use of all the methods.

25. Create a class Shape with attributes color and methods set_color() and get_color(). Create a subclass Rectangle that inherits from Shape and adds the attributes length and width, and methods calculate_area() and calculate_perimeter(). Create objects of both the classes and demonstrate the use of all the methods.
26. Create a class Animal with attributes name, species, and age, and methods get_name(), get_species(), get_age(), and make_sound(). Create a subclass Cat that inherits from Animal and adds the attribute breed, and method meow(). Create objects of both the classes and demonstrate the use of all the methods.
27. Create a class Person with attributes name, age, and gender, and methods get_name(), get_age(), get_gender(), and introduce(). Create a subclass Teacher that inherits from Person and adds the attribute subject, and method teach(). Create objects of both the classes and demonstrate the use of all the methods.
28. You are building a system to manage a restaurant's ordering and billing process. How would you use OOP to model the menu items, orders, and billing process? What classes and methods would you use to implement this system?
29. You are building a system to simulate a game of chess. How would you use OOP to model the game board, pieces, and player moves? What classes and methods would you use to implement this system?
30. You are building a system to manage a social media platform, with different types of users and their interactions. How would you use OOP to model the users, posts, and comments? What classes and methods would you use to implement this system?
31. You are building a system to manage a hotel's room reservations and occupancy. How would you use OOP to model the rooms, reservations, and occupancy status? What classes and methods would you use to implement this system?
32. You are building a system to manage a shopping cart for an e-commerce website. How would you use OOP to model the products, shopping cart, and checkout process? What classes and methods would you use to implement this system?

33. You are building a system to manage a music streaming service, with different types of songs and playlists. How would you use OOP to model the songs, playlists, and user interactions? What classes and methods would you use to implement this system?
34. You are building a system to simulate a flight booking system for an airline, with different types of flights and booking classes. How would you use OOP to model the flights, booking classes, and user interactions? What classes and methods would you use to implement this system?
35. You are building a system to manage a library of movies, with different genres and ratings. How would you use OOP to model the movies, genres, and user interactions? What classes and methods would you use to implement this system?
36. You are building a system to simulate a game of blackjack, with different card values and player strategies. How would you use OOP to model the deck of cards, player hands, and game outcomes? What classes and methods would you use to implement this system?
37. You are building a system to manage a parking garage, with different types of vehicles and parking spots. How would you use OOP to model the vehicles, parking spots, and user interactions? What classes and methods would you use to implement this system?
38.